

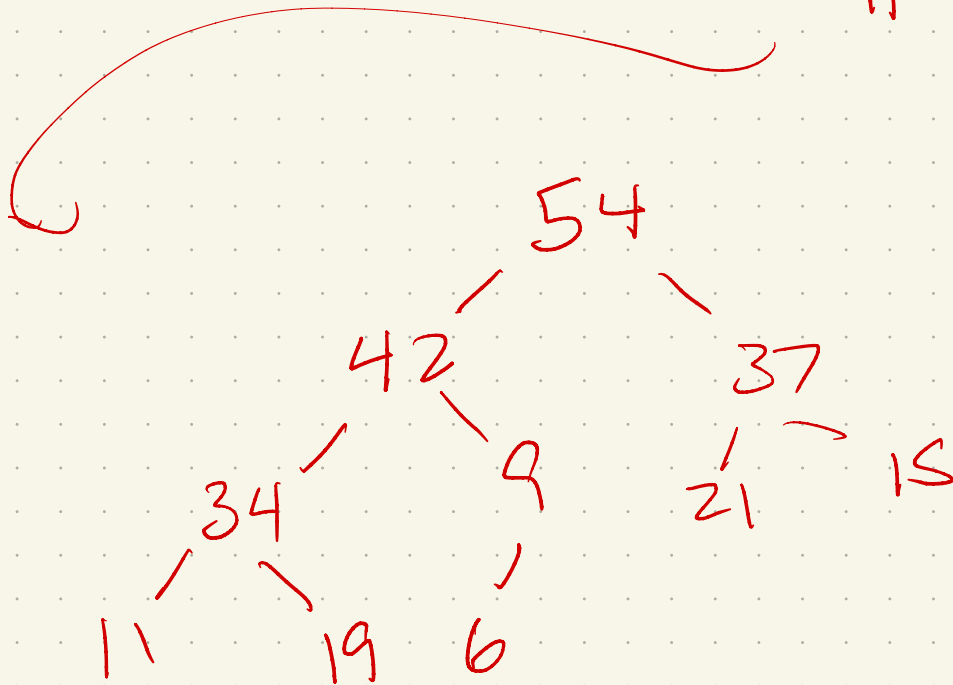
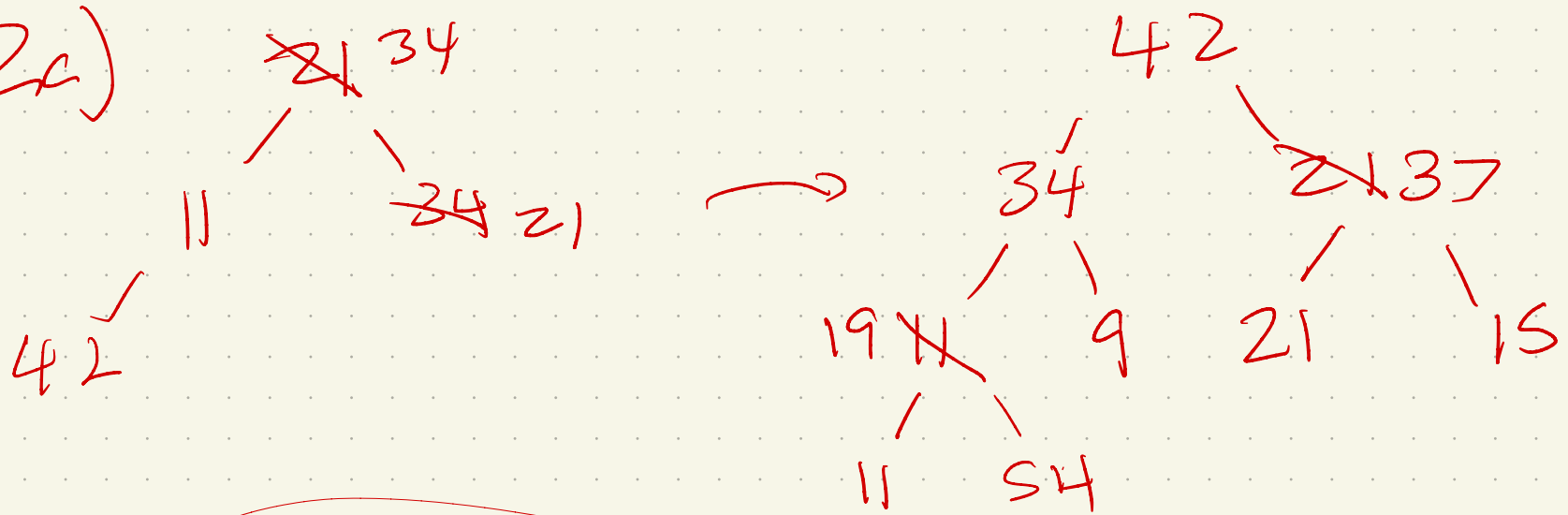
Transitions: DS + Algorithms

Binomial
Heaps



Last time: Heaps Worksheet

2c)



Why?

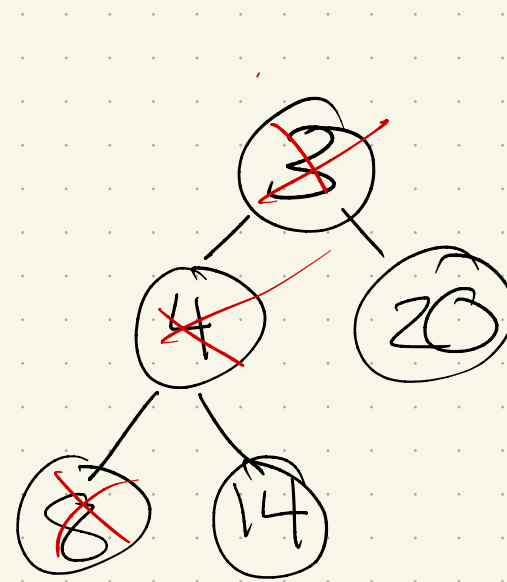
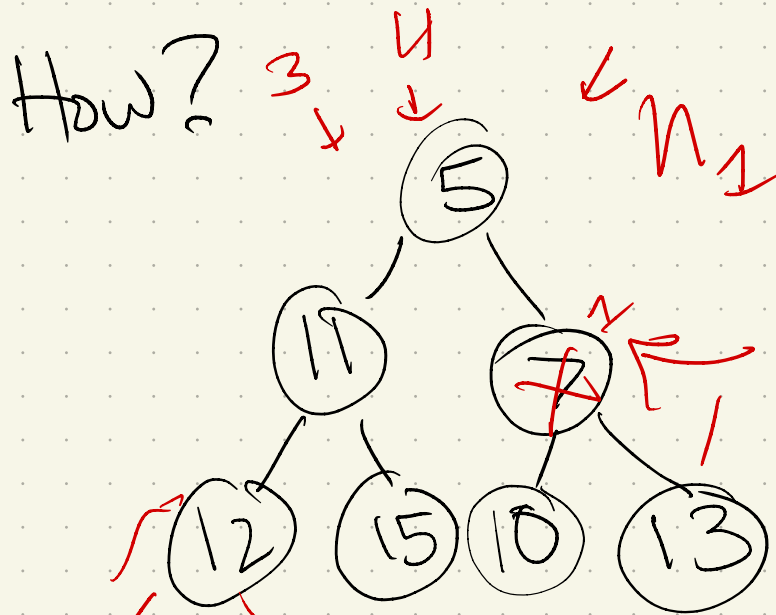
if use list or vector,
have $O(1)$ + $O(n)$
for main ops

heaps $\rightarrow O(\log n)$ for
all operations
to get/remove max
& insert

vs AVL: much better constants
used in finding shortest paths in a
(right after ~~best~~) graph

Another: Merge (H_1, H_2): ~~✱~~

Create a new heap with all values
of H_1 & H_2



$$n = n_1 + n_2$$

Runtime:
1. remove Min
↳ insert into H_1

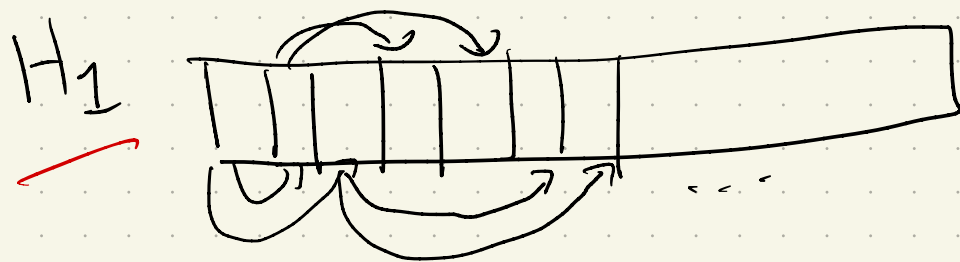
~~$$n_1 \log n_1 +$$~~

~~$$n \log n$$~~

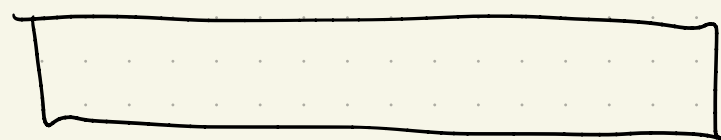
Improving?

Well $\rightarrow$ heapify!

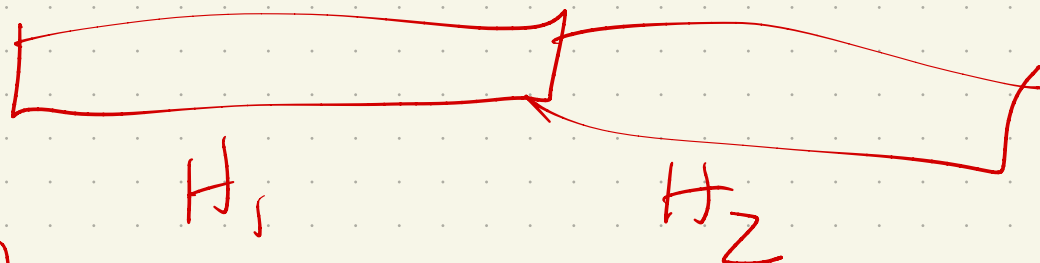
$O(n)$



H_2



Recall: heapify $\rightarrow$



& build heap

$\hookrightarrow O(n)$

Binomial Heap

Goal: Improve Merge ✓

$$\underline{O(n)} \xrightarrow{\text{heapify}} \underline{O(\log n)}$$

at the "cost" of min

$$O(1) \rightarrow O(\log n)$$

{But really not!!}

$O(n)$ to do n mins

Amortization: have lots of $O(1)$

cost extract min

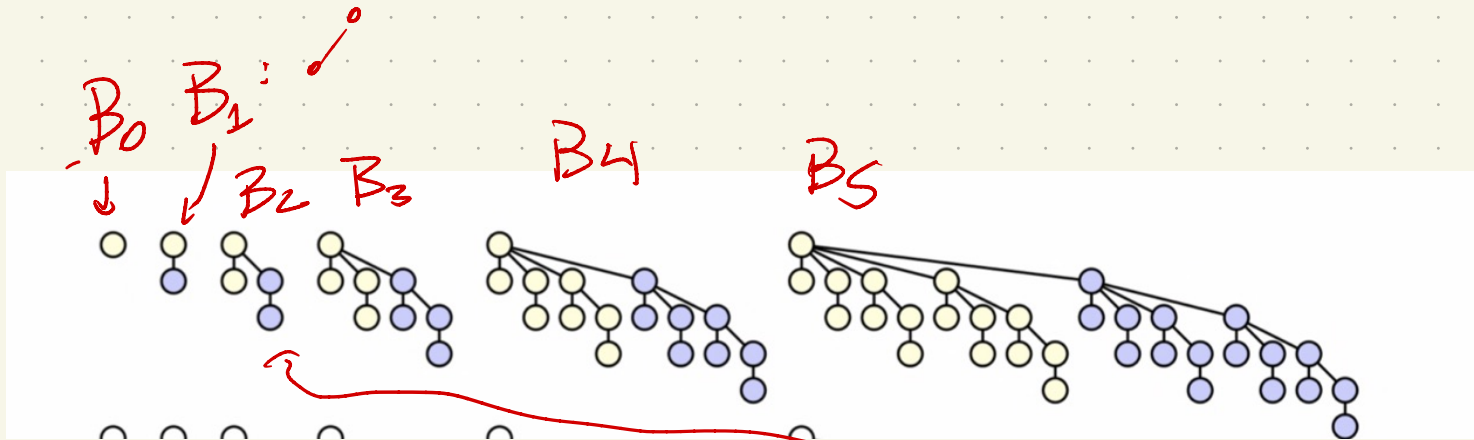
↳ but some slower

Dfn: A binomial tree, defined recursively:

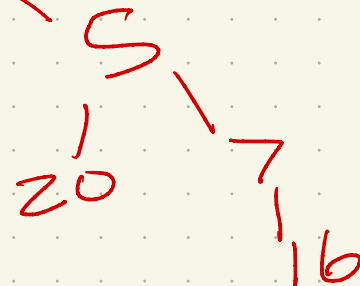
Base case: B_0

B_i : two copies of B_{i-1} , one root connected as (new) child of the other

Picture:

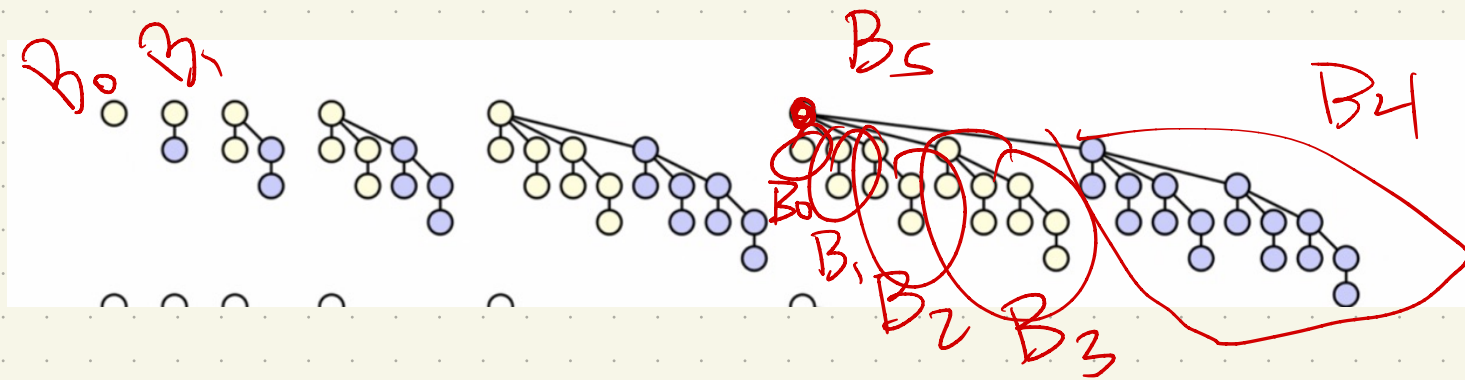
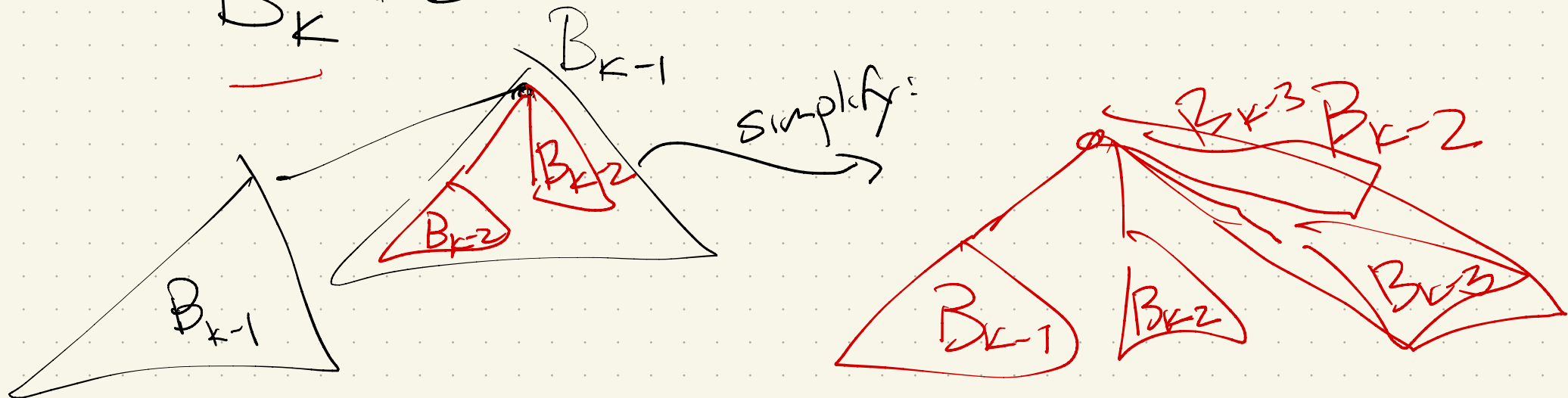


Later - heaps: B_2 :
(min heap)

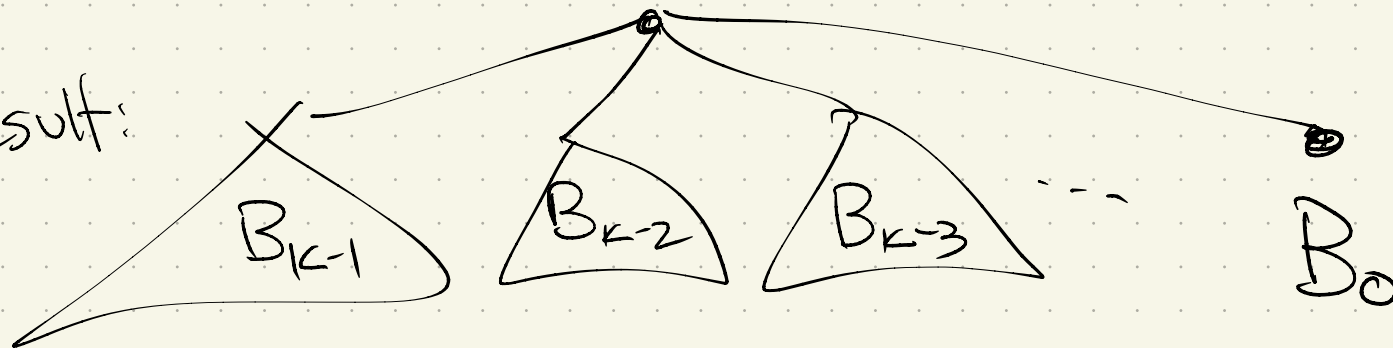


General pattern:

B_k : binomial tree of order k



End result:



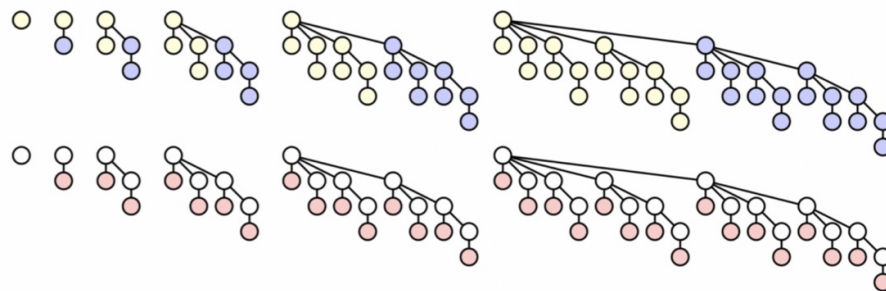
HWO question from previous semester!

4. A binomial tree of order k is defined recursively as follows:

- A binomial tree of order 0 is a single node.
- For all $k > 0$, a binomial tree of order k consists of two binomial trees of order $k - 1$, with the root of one tree connected as a new child of the root of the other. (See the figure below.)

Prove the following claims:

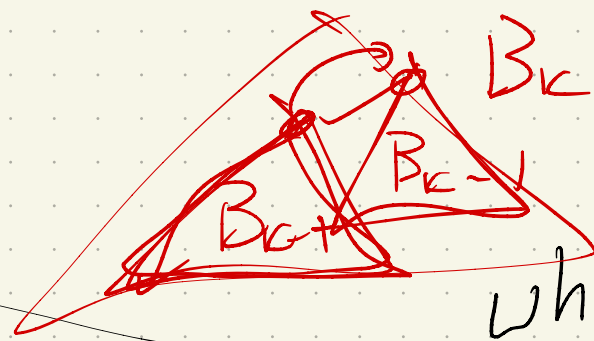
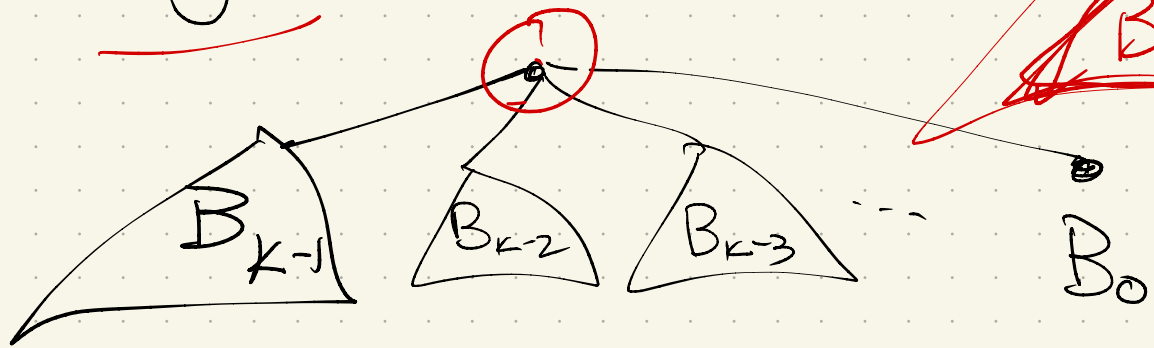
- [(a) For all non-negative integers k , a binomial tree of order k has exactly 2^k nodes.
- (b) For all positive integers k , attaching a leaf to every node in a binomial tree of order $k - 1$ results in a binomial tree of order k .
- (c) For all non-negative integers k and d , a binomial tree of order k has exactly $\binom{k}{d}$ nodes with depth d .



Binomial trees of order 0 through 5.
Top row: the recursive definition. Bottom row: the property claimed in part (b).

so size of $B^k = 2^k$ nodes

Height of $B(k)$:



which is biggest?

so $H(k) =$ height of ^{branch} tree of order k
 $= 1 + H(k-1) = 1 + 1 + H(k-2)$
 $= k$

In terms of # of nodes:

B_k has 2^k nodes & height k

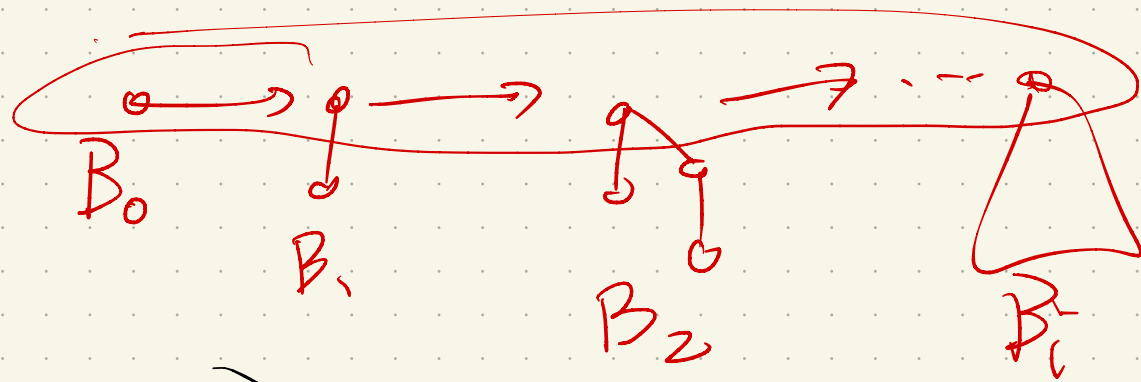
$\Rightarrow$ size $n = 2^k$, so
height $k = \log_2 n$

Binomial Heap (min heap)

- Like regular heap, child $>$ parent (for all nodes)
- But: this is a collection of binomial trees, with at most 1 of each size

so: one B_0
one B_1
one B_2

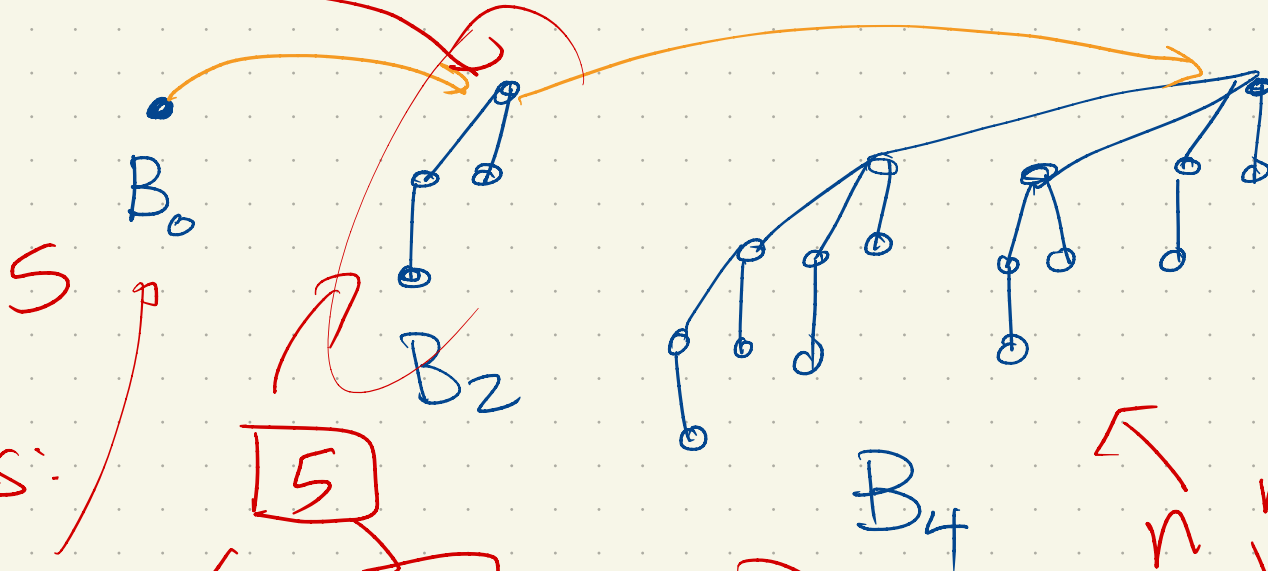
one B_i (some i)



Index via a linked list sorted by
degree $0 \dots i$

Ex List in orange trees in blue

Global
min



$n \leq 2^{i+1}$
 $\Rightarrow i+1 \leq \log n$

Keys:
 B_0

5
 12
 11

i is largest tree

n nodes in whole thing
 $\hookrightarrow$ list $\leq \log n$ items

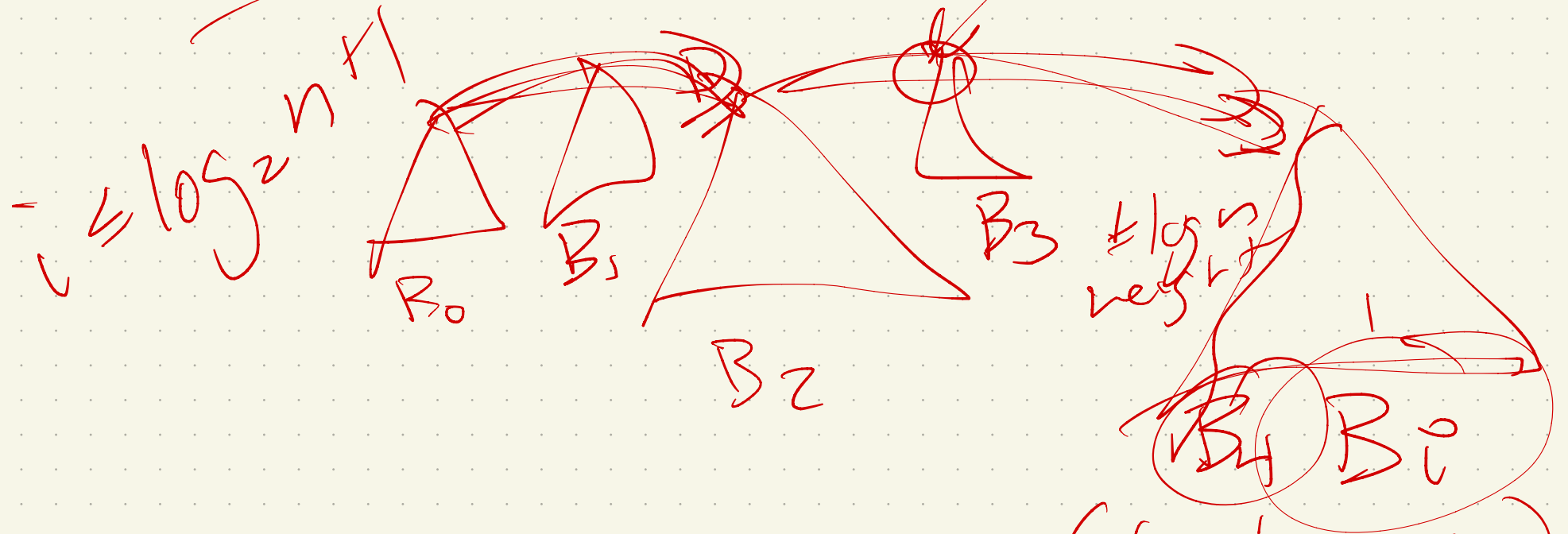
(note: no B_1 or B_3 in this example)

length of list (in terms of # of nodes):

n nodes tree of order $i = 2^i$ nodes

$$\# \text{ nodes} \leq 2^0 + 2^1 + \dots + 2^i = 2^{i+1} - 1$$

n nodes total



(last one)

$n \leq 2^0 + 2^1 + \dots + 2^i = 2^{i+1} - 1$

length of list: how big is i in terms of n ?

$n = 2^{i+1} - 1 \Rightarrow i = \log_2 n + 1$

length of list

How to search:

Look at the roots & take min

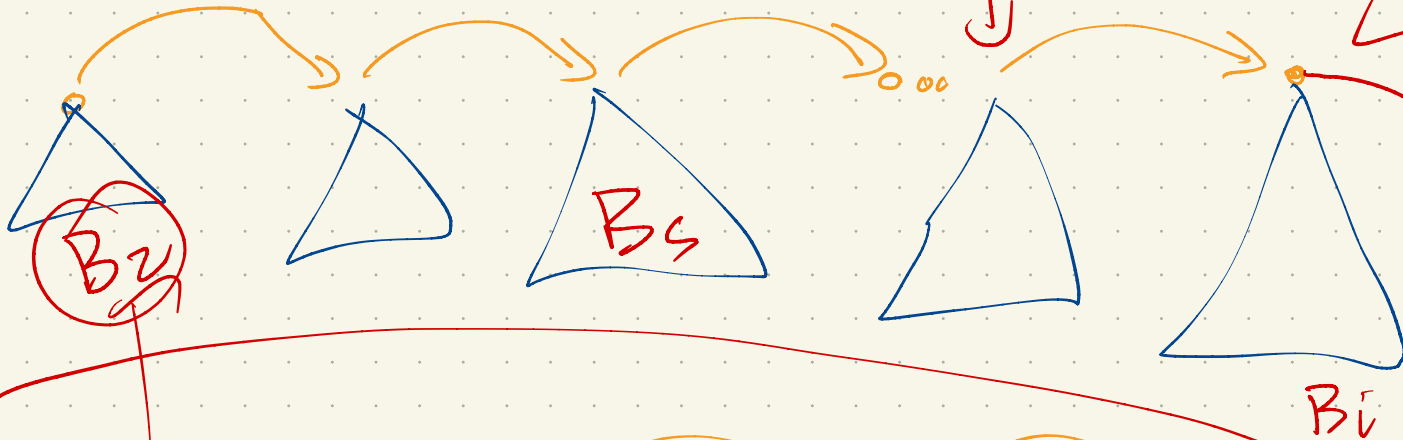
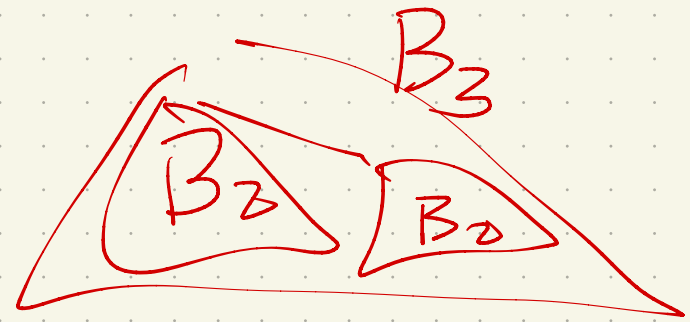
Runtime: $O(\log_2 n)$ if n nodes

But: can keep global ptr to minimum
(& just need to update it as you go)

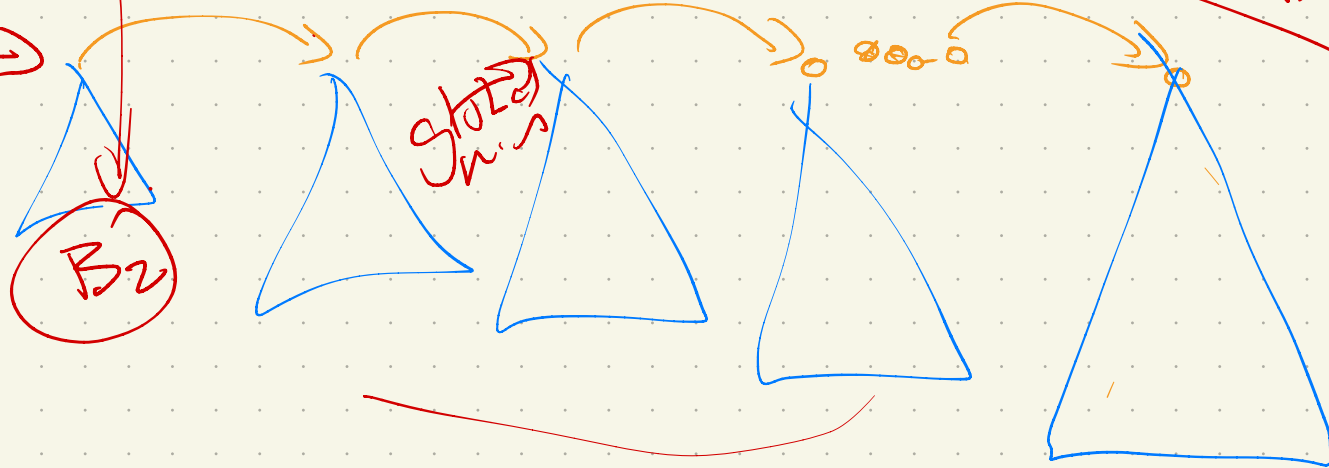
↳ Runtime: $O(1)$

Union (H_1, H_2):

global min



Heap 1



Heap 2

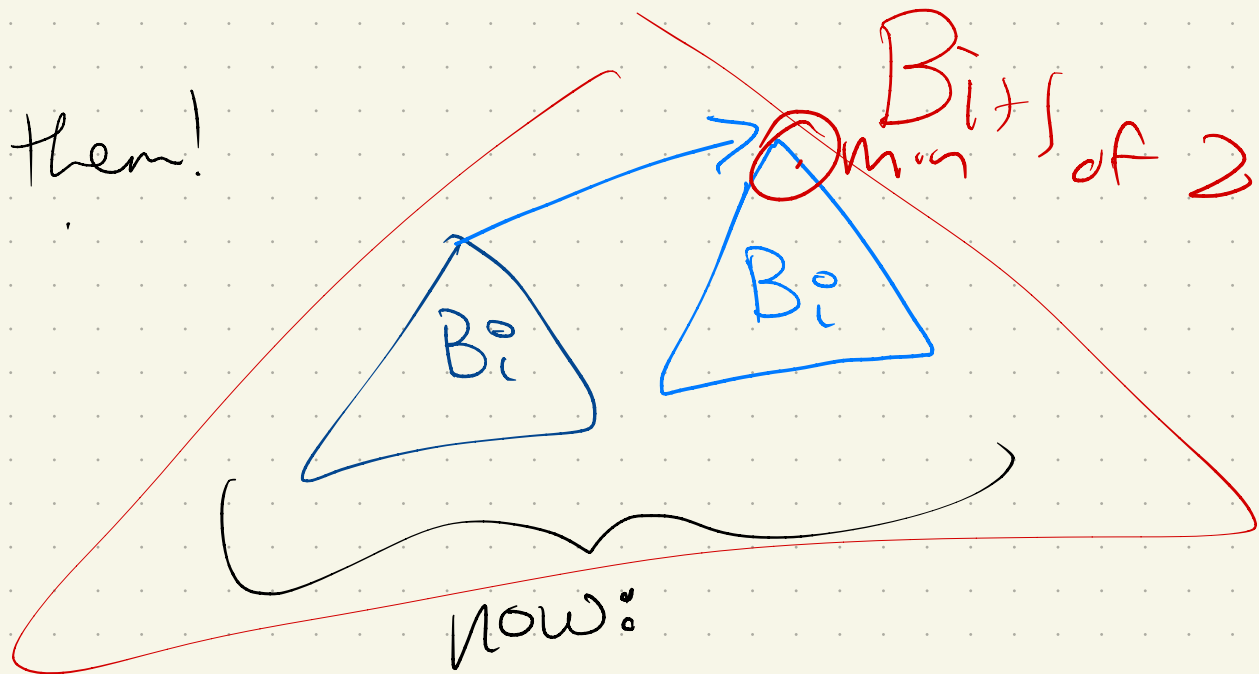
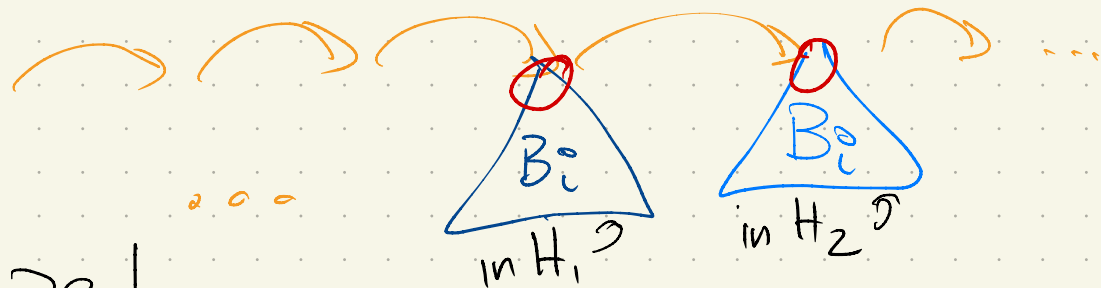
natural idea:

join them in bigger list

Problem: Merged list:

Could have
2 of same size!

Nice trick: combine them!



More detail:

$O(\log_2 n)$

for i from 0 to length of merged list:

if no nodes of degree i :



if 1 node of degree i

✓ keep it

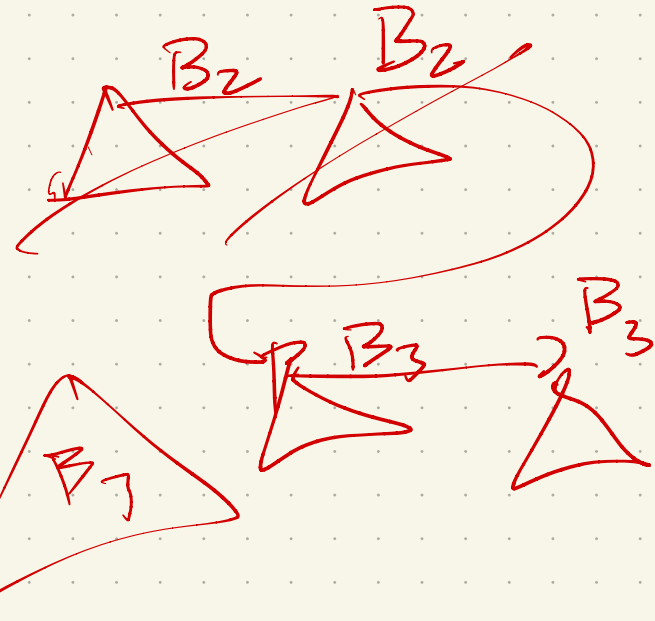
if 2 nodes of degree i

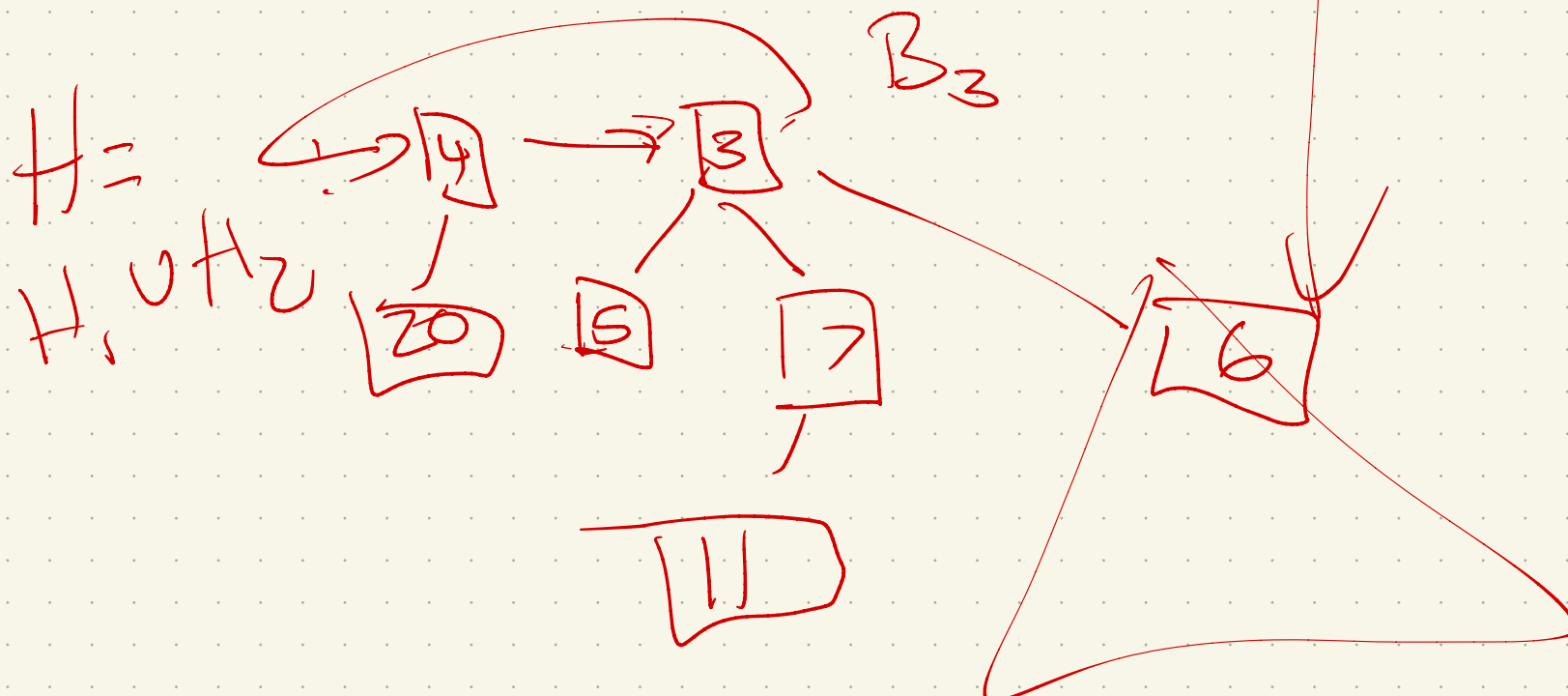
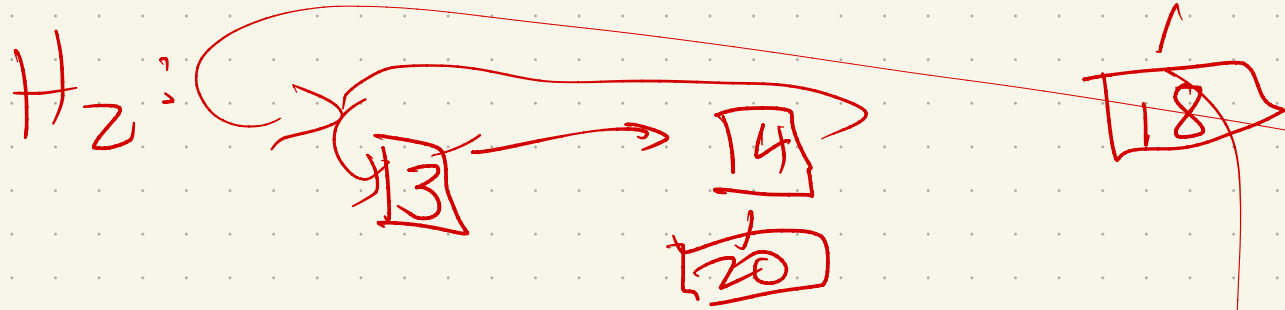
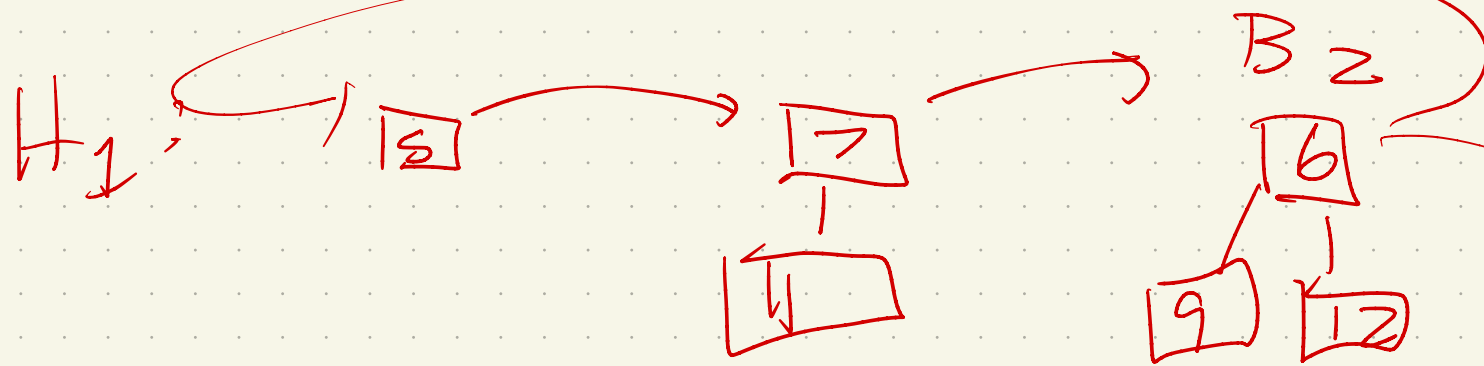
Combine: hang one with larger root
under other, and create a node
w/ deg $i+1$

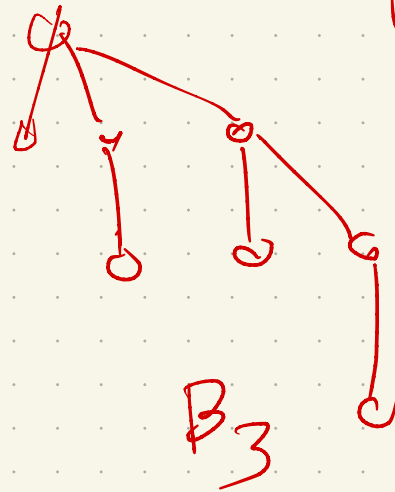
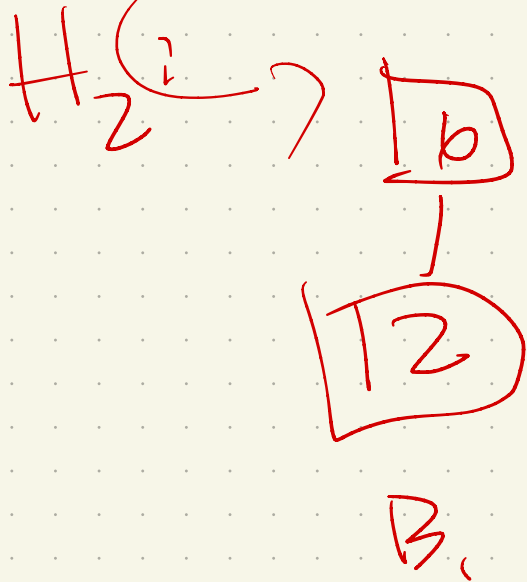
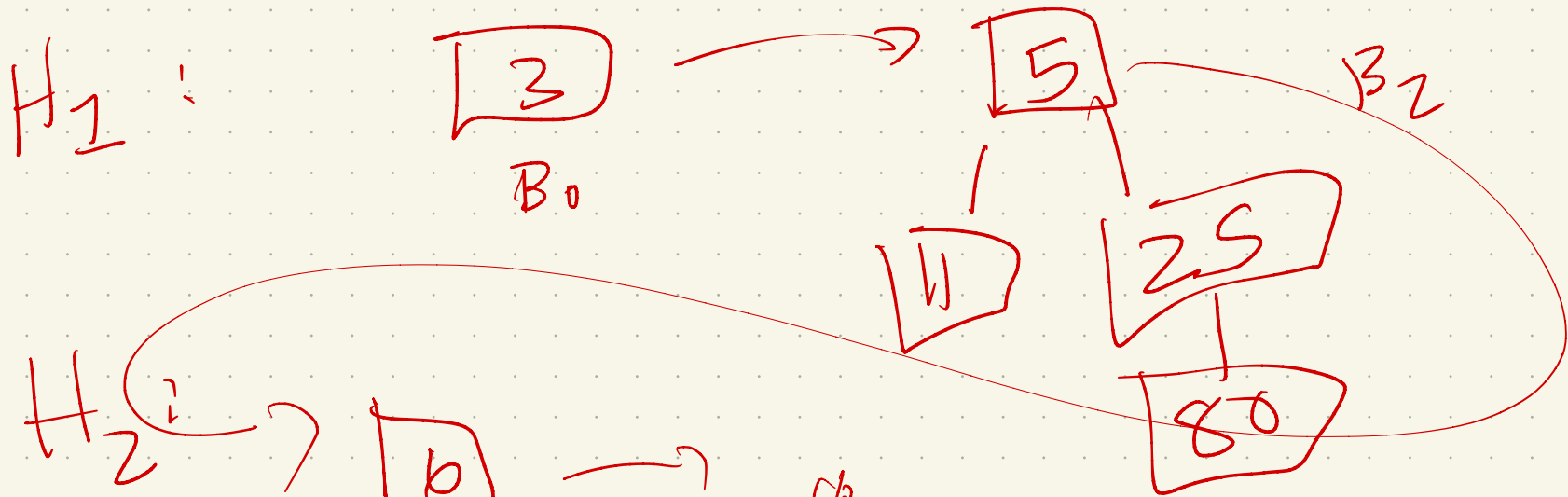
if 3 nodes of degree i

Combine 2 of them to deg $i+1$

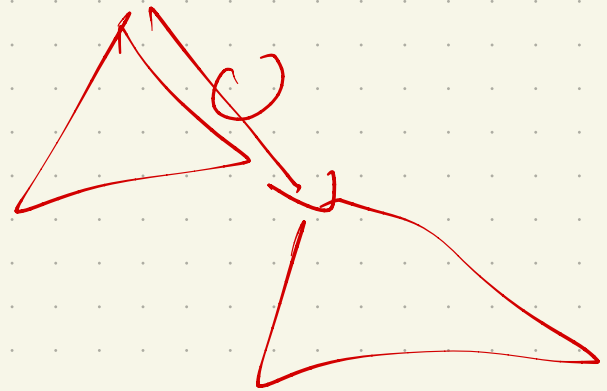
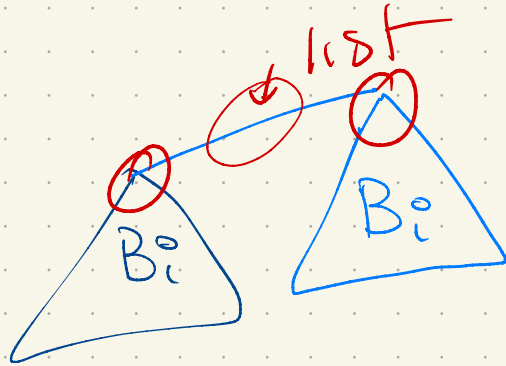
& keep one of deg i







Runtime of merge:
each internal merge:



Overall:

1 comparison
+ 2 ptr update
 $O(1)$

$\rightarrow O(1)$ time per node on
list $\Rightarrow O(\log_2 n)$ total

Insert(x, H)

Create a new binomial heap (size 1)

$\Rightarrow$ • (size 1 list)

B_0

Merge with H :

(keep pointer to global min)

Runtime: Worst case:

But: amortized insert is $O(1)$ time!

Why insert is faster:

Suppose we do n inserts, & consider a merge inside our loop:

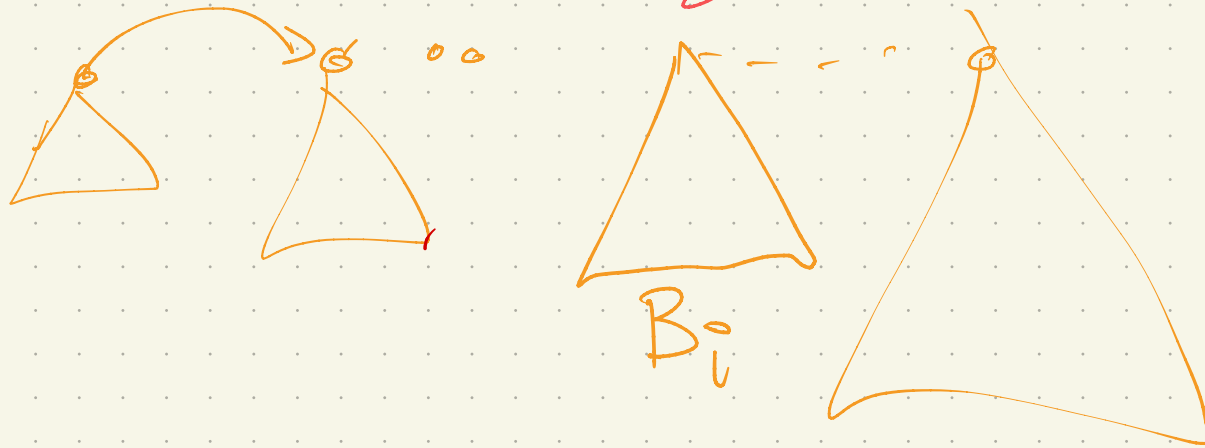
If no B_0 :

If $B_0 \dots B_i$ exist, & B_{i+1} does not:

Accounting method:

- ExtractMin():

Say we delete root of some B_i



Recall: B_i is what?

if we delete root:

So: flip children of root in B_i^0



Make a heap from these & merge with rest of the heap

Runtime:

Result :

	<u>Heap</u>	<u>Binomial heap</u>
get Min	$O(1)$	$O(\log n)$ $\rightarrow O(1)$ (w/ pointer)*
insert	$O(\log_2 n)$	$O(\log_2 n)$ + $O(1)$ amortized if n inserts
remove Min	$O(\log_2 n)$	$O(\log_2 n)$
decrease key	$O(\log_2 n)$	$O(\log_2 n)$
delete	$O(\log_2 n)$	$O(\log_2 n)$
union	$O(n)$	$O(\log_2 n)$

(* adds overhead to others, but only $O(1)$)

Only downsides:

constant factors!

Next time:

Hashing!

(ie Dictionaries in some languages)